



**Московский государственный технический университет
Факультет ИУ «Информатика и системы управления»
Кафедра ИУ-1 «Системы автоматического управления»**

ОТЧЕТ

по лабораторной работе №3

«Обработка границ, детектирование прямых и отрезков»

по дисциплине

«Методы и технологии распознавания образов»

Выполнил: Ахмад Яра

Группа: ИУ1И-31М

Отчет сдан: 09/12/2026

Оценка:

Часть 1

Сравните границы, которые находятся функциями:

E1=edge(Isobel');

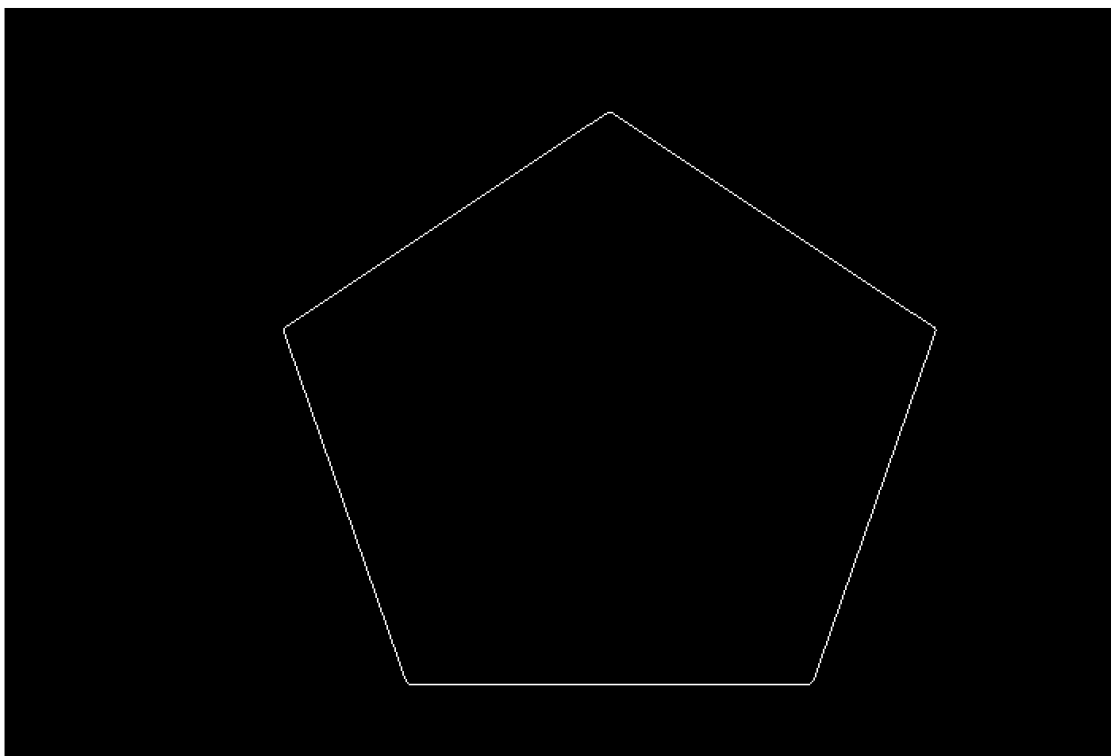
E2=edge(A, 'Canny');

В чем отличие этих вызовов? Чем обусловлены отличия результата?

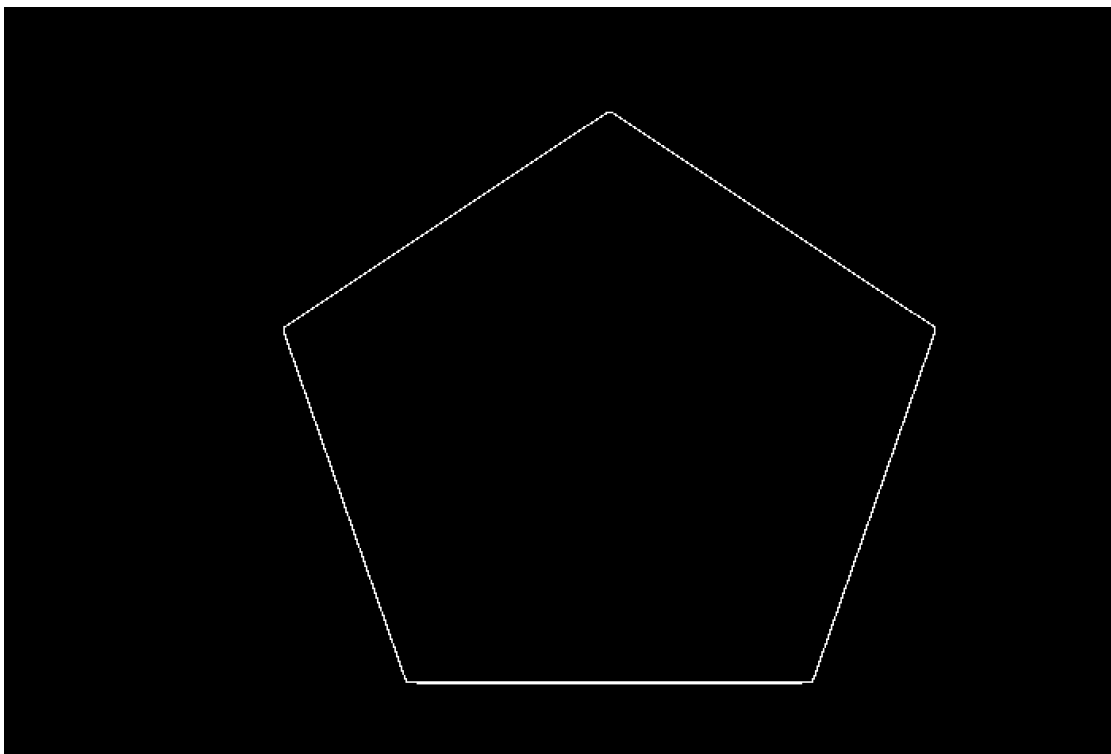
E1=edge(A,'Sobel');	E2=edge(A,'Canny');
Находит границы по величине градиента яркости изображения. Производные по координатам приближённо вычисляются с помощью оператора Собеля. Точки, в которых величина градиента превышает пороговое значение, рассматриваются как точки границы.	Метод Канни предварительно сглаживает изображение, вычисляет градиент яркости, выполняет подавление немаксимумов и использует два порога для выделения сильных и слабых границ. Слабые границы сохраняются, если они связаны с сильными. Поэтому метод Канни обычно лучше подавляет шум и формирует более тонкие и непрерывные границы.

```
A=imread('pentagon.png');  
  
[h,w,p]=size(A);  
if p==3,  
    A=rgb2gray(A);end  
E1=edge(A,'Sobel');  
figure,imshow(E1),title('Собель')  
E2=edge(A,'Canny');  
figure,imshow(E2),title('Канни')
```

Собель

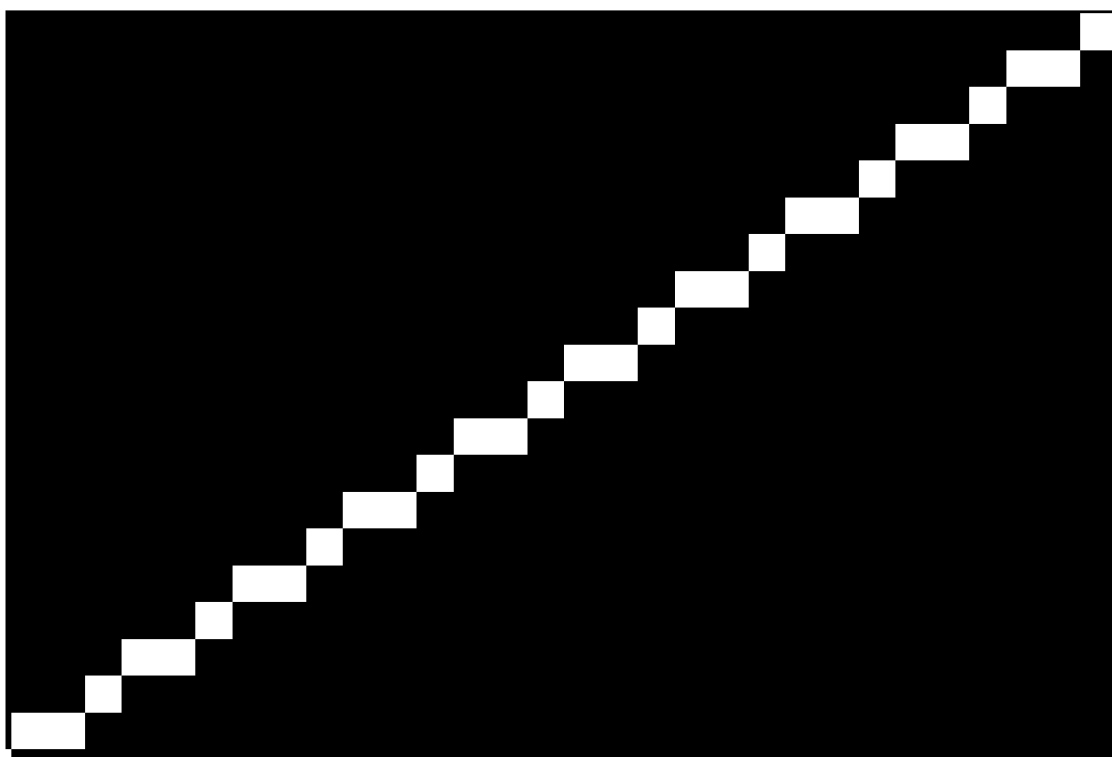


Канни

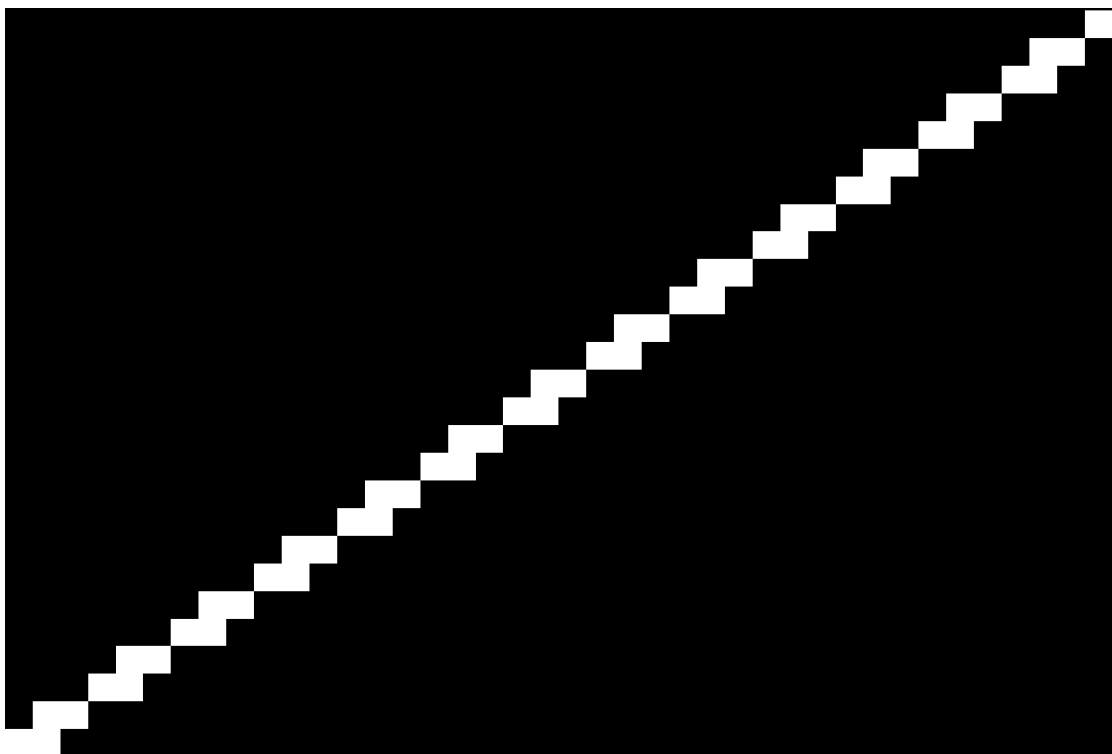


Верхний левый угол:

Собель



Канни



Нижний левый угол

Собель



Канни



Можно заметить, что метод Канни формирует более непрерывные и чёткие границы по сравнению с методом Собеля. Это связано с тем, что алгоритм Канни использует два пороговых значения: высокий порог для выделения сильных границ и низкий порог для обнаружения слабых границ. Слабые границы сохраняются только в том случае, если они связаны с сильными.

Кроме того, перед поиском границ метод Канни выполняет сглаживание изображения и подавление немаксимумов. Благодаря этому уменьшается влияние шума, а границы объектов получаются более тонкими, точными и непрерывными.

Часть 2

Почему в дуальном пространстве имеется только 4 максимума-бабочки? Куда делась еще одна?

В дуальном пространстве появляются только четыре максимума в форме «бабочки», поскольку стандартное уравнение прямой $y = kx + b$ не позволяет корректно описывать вертикальные прямые. Для вертикальной прямой угловой коэффициент k стремится к бесконечности, а свободный член (b) не определён. Для почти вертикальной прямой значения k и b могут стать больше допустимых границ пространства параметров Хафа. Поэтому пятая сторона пятиугольника не образует отдельного максимума в выбранном пространстве (k, b) .

```
KMAX = 1000;
BMAX = 1000;

kmax = 5;
bmax = 1000;

H_original = zeros(KMAX, BMAX);
tic;
for x = 1:w
    for y = 1:h
        if E(y, x) == 1
            for KI = 1:KMAX

                k = (KI-1)*(2*kmax)/(KMAX-1) - kmax;
                b = -x*k + y;
                BI = round((b+bmax)*(BMAX-1)/(2*bmax) + 1);

                if (BI >= 1) && (BI <= BMAX)

                    H_original(KI, BI) = H_original(KI, BI) + 1;

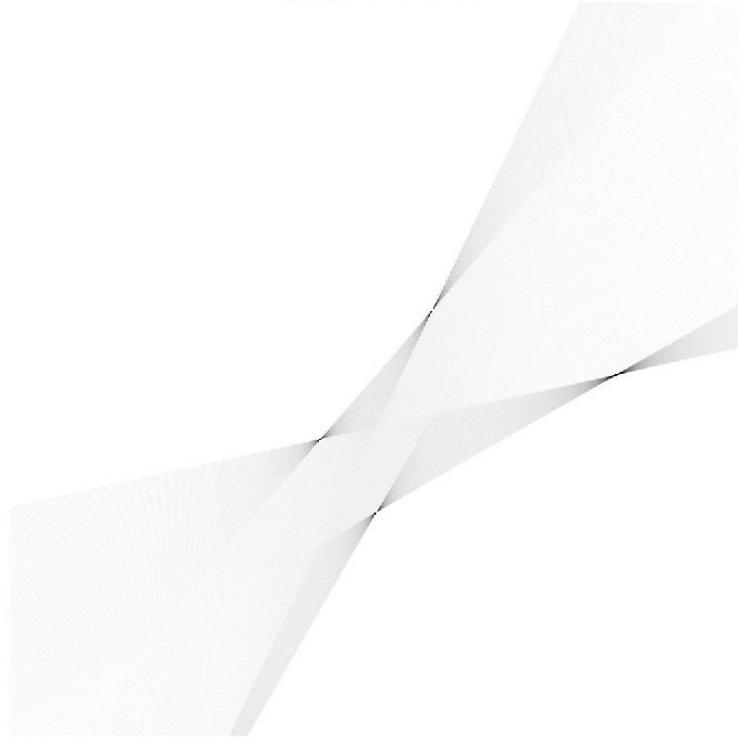
            end
        end
    end
end
```

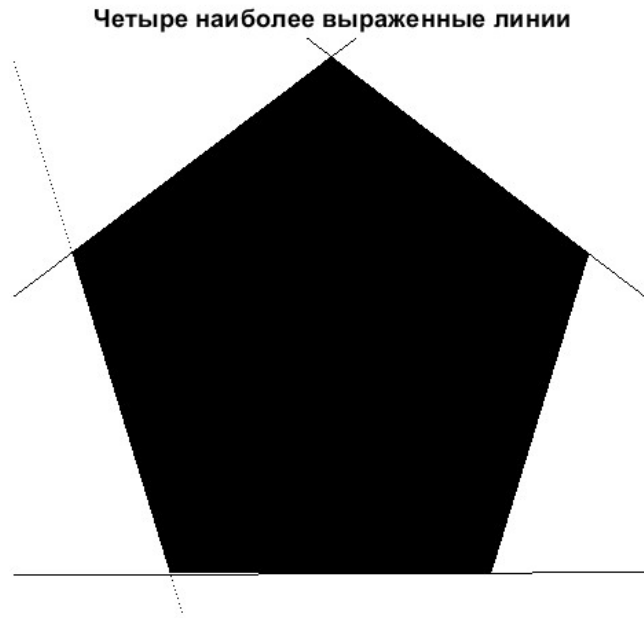
```

        end
    end
end
t_before = toc;
figure;
Hshow = H_original';
imshow(Hshow, [0 0.3*max(Hshow(:))]);
colormap(flipud(gray(256))); title('Пространство поиска');

```

Пространство поиска





Как ее найти?

Недостающую прямую можно найти, поменяв местами координаты x и y и выполнив повторный поиск, либо, в более общем случае, использовать нормальную форму представления прямой $\rho = x \cos \varphi + y \sin \varphi$ которая позволяет описывать прямые любой ориентации.

```
RMAX = round(sqrt(w^2 + h^2)) + 1;
FMAX = 360;

% Accumulator:
H_normal = zeros(RMAX, FMAX);

phi_values = 1:FMAX;
cos_phi = cosd(phi_values);
sin_phi = sind(phi_values);

tic;
for x = 1:w
    for y = 1:h
        if E(y,x) == 1

            for FI = 1:FMAX

                rho = x*cos_phi(FI) + y*sin_phi(FI);

                RI = round(rho) + 1;
```

```

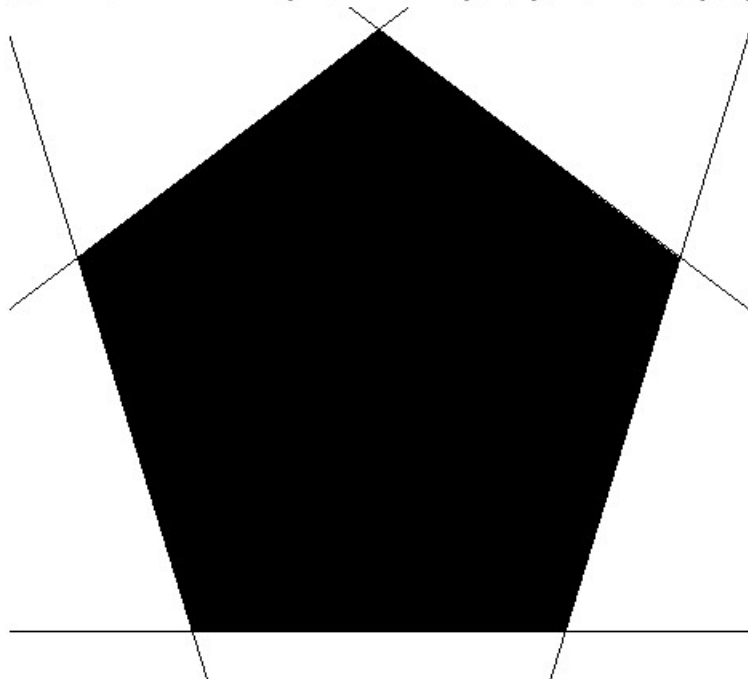
        if (RI >= 1) && (RI <= RMAX)
            H_normal(RI,FI) = H_normal(RI,FI) + 1;
        end
    end
end
end
end

t_normal = toc;
figure;
Hmax = max(H_normal(:));
imshow(-H_normal, [-0.25*Hmax 0]);
title('Пространство поиска');

```



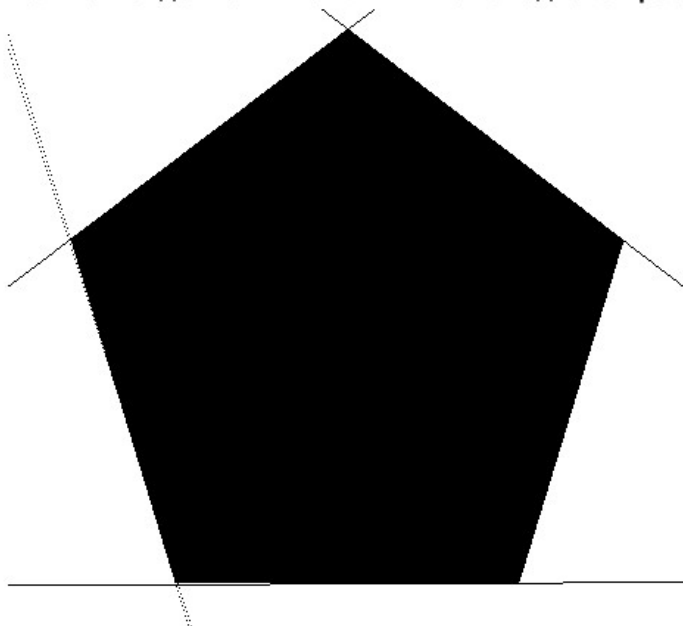
найденные линии в нормальном пространстве Хафа (θ, ρ)



Что произойдет, если строить пятую линию, для которой отсутствует максимум в дуальном пространстве?

Если попытаться построить пятую линию в исходном дуальном пространстве, алгоритм выберет наибольший из оставшихся откликов матрицы Хафа. Однако этот отклик не соответствует реальной пятой стороне пятиугольника, а представляет собой слабый или ложный максимум. Поэтому построенная пятая линия проходит в неправильном месте и не совпадает с контуром фигуры.

Принудительно найденная пятая линия из исходного пространства



Ускорьте алгоритм:

```
H_optimized = zeros(KMAX,BMAX);
alpha = (2*kmax)/(KMAX-1);
beta = (BMAX-1)/(2*bmax);
tic;
for x = 1:w
    for y = 1:h
        if E(y,x) ==
            for KI = 1:KMAX
                k = (KI-1)*alpha - kmax;
                b = -x*k + y;
                BI = round((b+bmax)*beta + 1);
                if (BI >= 1) && (BI <= BMAX)
                    H_optimized(KI,BI) = H_optimized(KI,BI) + 1;
                end
            end
        end
    end
end
t_after = toc;
```

Для ускорения алгоритма постоянные коэффициенты были вынесены за пределы вложенных циклов, чтобы не вычислять их многократно. После оптимизации время выполнения преобразования Хафа уменьшилось с **0.030603 с** до **0.021477 с**. Ускорение составило **1.425 раза**, что соответствует улучшению производительности примерно на **29.82%**.

До оптимизации = 0.030603 с

После оптимизации = 0.021477 с

Ускорение = 1.425 х

Улучшение = 29.82 %

Часть 3

Почему графики синусоид в дуальном пространстве отображены вверх ногами?

Синусоиды выглядят перевернутыми, потому что MATLAB отображает матрицы в системе координат изображения: первая строка находится сверху, а номер строки увеличивается вниз. В пространстве Хафа параметр (ρ) хранится как индекс строки в $H(R, \phi)$, поэтому увеличение (ρ) отображается вниз, а не вверх, как в обычной декартовой системе координат. Поэтому синусоидальные кривые выглядят перевернутыми по вертикали.

Модифицируйте алгоритм: перенесите начало координат в центр изображения. Убедитесь, что линии детектируются верно. Как изменится в этом случае дуальное пространство?

```
cx = (w + 1) / 2;
cy = (h + 1) / 2;

rho_max_center = ceil(sqrt((w/2)^2 + (h/2)^2));
RMAX_center = 2*rho_max_center + 1;
FMAX_center = 360;

H_center = zeros(RMAX_center, FMAX_center);

for x = 1:w
    for y = 1:h

        if E(y,x) == 1
            xc = x - cx;
            yc = y - cy;

            for FI = 1:FMAX_center

                rho = xc*cos_phi(FI) + yc*sin_phi(FI);
                RI = round(rho) + rho_max_center + 1;

                if RI >= 1 && RI <= RMAX_center
                    H_center(RI, FI) = H_center(RI, FI) + 1;
                end
            end
        end
    end
end
figure; imshow(-H_center, []);
title('Пространство Хафа с началом координат в центре изображения');
```



После переноса начала координат в центр изображения параметр (ρ) может принимать как положительные, так и отрицательные значения. Поэтому дуальное пространство становится более симметричным относительно $\rho = 0$. При использовании полного диапазона углов от 0° до 360° каждая прямая представляется дважды: как (ρ, φ) и как $(-\rho, \varphi + 180^\circ)$. В результате в дуальном пространстве появляются два симметричных набора синусоидальных кривых.

Ускорьте алгоритм: вынесите из цикла расчет синусов и косинусов (задайте таблицу синусов). Используя функции `tic` и `toc`, определите прирост производительности.

```
phi_values = 1:FMAX;  
  
cos_phi = cosd(phi_values);  
sin_phi = sind(phi_values);  
  
H_after = zeros(RMAX,FMAX);  
tic;
```

```

for x = 1:w
    for y = 1:h

        if E(y,x) == 1

            for FI = 1:FMAX

                rho = x*cos_phi(FI) + ...
                    y*sin_phi(FI);

                RI = round(rho) + 1;

                if RI >= 1 && RI <= RMAX
                    H_after(RI,FI) = ...
                        H_after(RI,FI) + 1;
                end
            end
        end
    end
end

t_trig_after = toc;

```

После вынесения вычислений синуса и косинуса за пределы внутреннего цикла и использования заранее рассчитанных таблиц время выполнения уменьшилось с 0.641087 с до 0.561380 с. Алгоритм стал примерно в 1.142 раза быстрее, что соответствует приросту производительности около 12.43%.

До оптимизации = 0.641087 с

После оптимизации = 0.561380 с

Ускорение = 1.142 раза

Прирост производительности = 12.43 %

Часть 4

Ускорьте алгоритм, используя информацию о направлении градиента яркости. Погрешность определения градиента принять равной 10 градусов. Используя функции tic и toc, определите прирост производительности.

Направление градиента перпендикулярно границе изображения и, следовательно, приблизительно соответствует направлению нормали обнаруженной линии. Вместо проверки всех возможных 360 углов Хафа для каждой точки края, рассматривались только углы в пределах $\pm 10^\circ$ от направления градиента. Также проверялось противоположное направление градиента, поскольку нормаль может иметь две ориентации. Это значительно сократило количество операций с накопителем и, следовательно, уменьшило время выполнения.

```
H_gradient = zeros(RMAX,FMAX);
angleTolerance = 10;
tic;

for x = 1:w
    for y = 1:h
        if E(y,x) == 1

            g = mod(round(gamma(y,x)),360);
            centers = [g, mod(g+180,360)];

            for C = 1:2

                centerAngle = centers(C);

                for delta = -angleTolerance:angleTolerance
                    angle = mod(centerAngle + delta,360);

                    if angle == 0
                        FI = 360;
                    else
                        FI = angle;
                    end
                    rho = x*cos_phi(FI) + y*sin_phi(FI);
                    RI = round(rho) + 1;
                    if RI >= 1 && RI <= RMAX

                        H_gradient(RI,FI) = H_gradient(RI,FI) + 1;
                    end
                end
            end
        end
    end
end
```

```
end  
end  
  
t_gradient_after = toc;
```

В результате время выполнения уменьшилось с **0.007117 с** до **0.002472 с**. Ускорение составило **2.878 раза**, что соответствует **улучшению производительности на 65.26%**. Это показывает, что использование направления градиента эффективно сокращает количество операций с аккумулятором при сохранении корректного обнаружения линий.

До оптимизации = 0.007117 с
После оптимизации = 0.002472 с
Ускорение = 2.878 раза
Улучшение производительности = 65.26 %

Пространство Хафа с использованием направления градиента

